

README

docs/coverage/ — CONST-048 Four-Layer Coverage Ledger Suite

Round 68 creation — closes 50+ round deferred governance debt for the CONST-048 submodule × feature × platform × invariant ledger. Complements the pre-existing feature-axis ledger.md (round 41+) with a submodule-axis, fine-grained view.

What lives here

File	Purpose	Edited by
COVERAGE_LEDGER.md	The main ledger. Row = (submodule, feature, platform); columns = the 6 CONST-048 invariants + Overall + Notes.	Operator (hand-marks PASS) + generator script (refreshes scaffold)
SCHEMA.md	Row format, status vocabulary, cross-reference points, invariant definitions, regeneration rules.	Operator on schema changes
README.md	This file — explains the suite.	Operator
ledger.md	Pre-existing feature × invariant rollup (round 41+). Complementary, NOT superseded.	scripts/regenerate-coverage-ledger.sh
decoupling_review.md	CONST-051(B) decoupling audit.	Operator
rename_safety_guardrails.md	CONST-052 rename-safety capture.	Operator

Why two ledgers?

ledger.md (round 41) answers: “*For each user-facing feature F01..F30, what is the test posture across all six invariants?*” — useful for release-readiness sweeps focused on the end-user product surface.

COVERAGE_LEDGER.md (round 68) answers: “*For each owned-org submodule we ship, what features does it expose, and what is the test posture per platform per invariant?*” — useful for CONST-051(A) equal-codebase audits (does HelixSpecifier get the same engineering attention as helix_code/?) and CONST-047 recursive-application audits.

Both ledgers **MUST** stay in sync with reality — divergence between them surfaces gaps in either the feature roster (user manual) or the submodule roster (docs/improvements/submodule_owned.txt).

CONST-048 invariants (verbatim from constitution §11.4.25)

#	Invariant	Sourced from
I1	Anti-bluff posture with captured runtime evidence	CONST-035 / Article XI §11.9
I2	Proof of working capability end-to-end on target topology (no mocks beyond unit tests)	CONST-050(A)
I3	Implementation matches the documented promise	§11.4.12
I4	No open issues / bugs surfaced	Issues.md / Issues_Summary.md cross-check
I5	Full documentation in sync	§11.4.12
I6	Four-layer test floor (pre-build + post-build + runtime + paired mutation)	§11.4 paired-mutation discipline

Regeneration cadence

Triggers (per constitution §11.4.25 release-gate sweep): 1. **Per-round close-out** — the round agent appends evidence rows for the submodule it touched. 2. **Pre-release sweep** — operator runs `bash scripts/generate-coverage-ledger.sh` to refresh the scaffold + verify every owned submodule has at least one row (exit non-zero if missing). 3. **Post constitution/ pull** (CONST-055) — re-run the generator to surface any new submodule added to submodule_owned.txt.

Status vocabulary (one-line summary; full table in SCHEMA.md)

Symbol	Meaning
PASS	Captured runtime evidence in current cycle
PENDING_FORENSICS:	Work landed but evidence not yet pasted into the ledger
OPERATOR-BLOCKED:	Needs operator action (real LLM key, hardware, network) — §11.4.21 audit applies
UNCONFIRMED:	Not yet audited / no evidence captured
SKIP-OK: #<ticket>	Intentionally not covered, with documented reason
PARTIAL	Covered for some platforms / scenarios but not all — see Notes
N/A	Invariant does not apply to this row

Anti-bluff guarantee (CONST-035 / Article XI §11.9): default cell status is UNCONFIRMED: — never PASS. PASS marker requires either (a) a CONTINUATION.md round reference quoting the captured evidence, or (b) an in-ledger evidence snippet pasted in the Notes column. PASS without one of these is a §11.4 PASS-bluff at the governance layer, severity equivalent to a false-success test result.

Cross-reference points (per SCHEMA.md §3)

- **CONTINUATION.md** — round-by-round narrative is the canonical source for PASS justifications.
- **Issues.md / Issues_Summary.md** — feeds I4 (no-open-bugs). HelixCode does not currently maintain a root-level Issues tracker; cross-check is via `grep of // TODO(round-N) : // BUG #NN`: markers in source.
- **SKIP-OK markers** in `*_test.go` — feeds I6 (mutation-test partial-coverage signals).
- `docs/improvements/submodule_owned.txt` — canonical roster of owned submodules; one row per submodule minimum.
- **CONST-051(C) layout audit** in `ledger.md` — feeds I3 (decoupling matches the documented promise of project-not-aware reuse).

Initial population approach (round 68)

Per the round 68 brief and CONST-035 anti-bluff discipline: the ledger is populated **conservatively**.

- PASS markers landed ONLY where CONTINUATION.md round narrative documents captured evidence ($\approx$ a dozen submodule $\times$ feature rows from rounds 23, 37, 41, 53, 60, 62, 63, 64, 65).
- Every other cell is UNCONFIRMED: until a future round captures evidence and promotes it.
- Auto-populating every cell PASS would itself be a CONST-035 PASS-bluff at the governance layer — explicitly forbidden by the brief.

Subsequent rounds promote cells UNCONFIRMED: $\rightarrow$ PASS ONLY by adding (a) the CONTINUATION round reference, AND (b) the captured evidence path/snippet in Notes. The generator NEVER auto-promotes.

Release-gate test runner (round 74)

`scripts/release-gate-test.sh` is the EXECUTION counterpart to `generate-coverage-ledger.sh`. Where the ledger tracks STRUCTURAL coverage (what we believe is tested, with captured evidence), the release-gate runner ACTUALLY EXECUTES tests against every owned-org submodule and reports an honest aggregate signal.

What it covers

- Walks every submodule listed in `docs/improvements/submodule_owned.txt` (round 56 canonical roster).
- For each submodule with a `go.mod`, runs `GOMAXPROCS=2 nice -n 19 go test -count=1 -race -timeout=180s ./...` inside the submodule root.
- Captures stdout+stderr to per-submodule log files under `$RELEASE_GATE_LOG_DIR` (default `/tmp/release-gate-<pid>/`).
- Parses PASS/FAIL/SKIP line counts.

- Detects bare `--- SKIP:` lines without `SKIP-OK:` markers — these are CONST-035 skip-bluff violations.

What it does NOT cover

- The inner `helix_code/` Go module — that’s covered by `make test-full` per CLAUDE.md §3.4 (its own four-layer floor with real PostgreSQL + Redis + Ollama infra).
- Non-Go submodules (`assets/`, `github_pages_website/`) — these are SKIP-NO-GOMOD, legitimately untested by this runner. Their coverage lives elsewhere (the asset/static-site pipelines).
- Integration / E2E tests requiring docker-compose — those need `make test-infra-up` per submodule.
- Mutation tests — paired-mutation runs are tracked in `COVERAGE_LEDGER.md` cell-by-cell.

Exit code semantics

Exit	Meaning
0	All owned submodules either PASSEd (<code>go test</code> succeeded) or were legitimately SKIP-NO-GOMOD. Release gate GREEN. With <code>--skip-env-failures</code> , also returned when ONLY ENV-CLASS failures occurred and were skipped.
1	At least one submodule FAILEd OR at least one bare SKIP detected. Release gate RED — see the human summary or <code>--json</code> output for the failing list. (Round 74 default behaviour, preserved when <code>--skip-env-failures</code> not set.)
2	Invalid flag, missing required file, or <code>--check</code> self-validation failed.
3	Round 89 — mixed: at least one LOGIC-CLASS FAIL plus at least one skipped ENV-CLASS FAIL. Only emitted when <code>--skip-env-failures</code> is set. Tells CI “logic-broken AND env-dirty” so the operator can distinguish from a clean run that simply skipped some env churn.

Flags

Flag	Effect
<code>--json</code>	Emit machine-readable JSON summary for CI integration. Includes per-submodule + aggregate counts + <code>log_dir</code> path + (round 89) <code>fail_class</code> + <code>fail_reason</code> + <code>n_env_class_fail</code> + <code>n_logic_class_fail</code> .
<code>--quick</code>	Stop on first FAIL. Useful for fast CI signal where any failure should block. Under <code>--skip-env-failures</code> , only LOGIC-CLASS FAIL triggers early-stop (ENV-CLASS is non-blocking).

Flag	Effect
<code>--only=<glob></code>	Restrict to submodules matching the shell glob (e.g. <code>--only='dependencies/vasic-digital/*'</code>). Useful for targeted re-runs.
<code>--check</code>	Self-validate script + paths without running real tests. Round 89 extends this to exercise the env-vs-logic classifier against five synthetic fixtures (3 ENV, 2 LOGIC) and assert correct classification.
<code>--skip-env-failures</code>	Round 89 — classify each FAIL as ENV-CLASS (operator-fixable; <code>go mod tidy / install dep</code>) vs LOGIC-CLASS (genuine code defect). Report ENV-CLASS but treat as non-blocking. LOGIC-CLASS always blocks. Use day-to-day; OMIT for release gates.

Round 89 — env-vs-logic FAIL classification

Round 74 reports raw FAIL counts. Round 89 layers a deterministic regex-based classifier on top to distinguish:

- **ENV-CLASS FAIL** — caller environment is missing something (`go mod tidy` hasn't run, system header/library absent, executable not on PATH, port permission denied). Operator-actionable; not a code defect.
- **LOGIC-CLASS FAIL** — genuine test-logic or production-code defect. Always blocks.

Detection patterns (env-class — ANY hit reclassifies the FAIL as ENV):

Pattern (extended grep)	Operator action
missing <code>go\sum</code> entry	<code>go mod tidy</code>
cannot find package	<code>go mod tidy</code>
no required module provides	<code>go mod tidy</code>
updates to <code>go\mod</code> needed	<code>go mod tidy</code>
inconsistent vendoring	<code>go mod vendor</code>
package <code>[^]+</code> is not in std	<code>go mod tidy / check Go version</code>
command not found	install missing tool
executable file not found	install missing tool (e.g. chromium for chromedp)
permission denied	<code>chmod / chown / use non-privileged port</code>
fatal error: <code>.*\h</code> : No such file	install -dev package for the C header
<code>X11/[A-Za-z]+\h</code>	install <code>libX11-dev</code> / equivalent
<code>Xcursor/Xcursor\h</code>	install <code>libxcursor-dev</code>
<code>gtk/gtk\h</code>	install <code>libgtk-3-dev</code>

Pattern (extended grep)	Operator action
cannot find -l[A-Za-z]	install matching system library

Anti-bluff invariant: classification is deterministic regex — **no LLM grading**. Ambiguous patterns fail-closed to LOGIC-CLASS, never silently downgraded to ENV. Per-failure output includes the matched pattern (or “no env-pattern match — fail-closed”) so the classification is auditable + reproducible.

Why this matters: round 74 reported 26 FAILs across owned submodules. Rounds 72/74/87 forensics confirmed most were environmental (caller hadn’t run `go mod tidy`, missing system libs, etc.) — not genuine code defects. Without the classifier, CI cannot distinguish “operator needs to run `go mod tidy`” from “developer wrote broken code”. Round 89 makes that distinction mechanical.

Recommended invocation pattern

```
# Day-to-day CI (env churn non-blocking, logic defects still
    blocking):
bash scripts/release-gate-test.sh --skip-env-failures --json >
    release-gate.json
case $(jq '.exit_code' release-gate.json) in
    0) echo "green (possibly some ENV skipped)" ;;
    1) echo "LOGIC-CLASS failure – block merge" ;;
    3) echo "LOGIC-CLASS failure AND env-dirty – block merge +
        ask operator to tidy" ;;
esac

# Release gate (strict – every FAIL blocks, including env):
bash scripts/release-gate-test.sh

# Manual pre-release sweep (full, human-readable, with
    classification):
bash scripts/release-gate-test.sh --skip-env-failures

# Targeted re-run after operator runs `go mod tidy` to clear ENV-
    CLASS:
bash scripts/release-gate-test.sh --only='submodules/cache'

# Sanity check that the script + classifier are healthy (no real
    test runs):
bash scripts/release-gate-test.sh --check
```

CI integration recipe: use `--skip-env-failures` in the day-to-day per-PR gate so a teammate who hasn’t run `go mod tidy` doesn’t redden the board for everyone. Use the **bare** `release-gate-test.sh` (no flag) in the release-tag gate so env churn cannot ship past a tagged release. The two gates answer different questions: “is the code OK?” vs “is the WORLD ready to ship?”.

Cross-reference with the coverage ledger

The two tools answer different questions about CONST-048 invariant 6:

- `COVERAGE_LEDGER.md` answers “*do we BELIEVE this submodule × platform combo has captured runtime evidence?*” (structural state).
- `release-gate-test.sh` answers “*does go test ./... ACTUALLY pass for this submodule RIGHT NOW?*” (live execution).

When the ledger says PASS for invariant I6 on a submodule but `release-gate-test.sh` reports FAIL for that same submodule, the ledger is stale — a §11.4 PASS-bluff at the governance layer. Operator demotes the cell to `PENDING_FORENSICS:` or `OPERATOR-BLOCKED:` until reconciled.

CONST-048 invariant 6 — closure note (round 74)

Round 68 landed the STRUCTURAL view of invariant 6. Round 74 lands the EXECUTION view. Together they constitute CONST-048 invariant 6’s meta-repo-level closure. The inner-module level was already covered by `make test-full` per CLAUDE.md §3.4 + the round 41 feature-axis ledger.md.

The verbatim 2026-05-19 operator mandate that motivated round 74 (preserved per CONST-049 §11.4.17 in the script header):

“all existing tests and Challenges do work in anti-bluff manner - they MUST confirm that all tested codebase really works as expected! We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can’t be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!”

See also

- `scripts/release-gate-test.sh` — programme-wide release-gate test runner (round 74; execution counterpart)
- `scripts/generate-coverage-ledger.sh` — scaffold generator + violation checker (round 68; structural counterpart)
- `scripts/regenerate-coverage-ledger.sh` — pre-existing feature-axis regenerator (round 41+)
- `docs/coverage/ledger.md` — pre-existing feature-axis ledger
- `docs/coverage/SCHEMA.md` — full schema documentation

Sources verified 2026-05-29: internal-governance doc — no third-party-service operator instructions to cross-reference. This README documents HelixCode's own CONST-048 coverage-ledger suite and the scripts/release-gate-test.sh/scripts/generate-coverage-ledger.sh tooling; the only external references are Go toolchain commands (go test, go mod tidy, go mod vendor) and a chromedp/chromium mention in the env-class-FAIL classifier — none carry a version pin requiring a vendor-docs check, and the Go commands are stable, long-standing subcommands. Version authority per CLAUDE.md §3.1 (confirmed in-tree): inner module go 1.26, root go 1.25.2; PostgreSQL 15+; Redis 7+ — no stale build/Docker version pins in this doc to correct. Per CONST-036, any model/provider identifiers are LLMsVerifier-sourced at runtime (helixcode llm models list), not pinned here. Reviewed against the live tree on this date; no corrections required.